LA-UR-26-21433

Approved for public release; distribution is unlimited.

Title: Kessel: Continuous Integration and Developer Workflows

Author(s): Berger, Richard Felix

Intended for: HPSF Conference 2026, 2026-03-16/2026-03-20 (Chicago, Illinois, UNITED STATES)

Issued: 2026-03-09 (rev.1)

Los Alamos
NATIONAL LABORATORY

Los Alamos National Laboratory, an affirmative action/equal opportunity employer, is operated by Triad National Security, LLC for the National Nuclear Security Administration of U.S. Department of Energy under contract 89233218CNA000001. By approving this article, the publisher recognizes that the U.S. Government retains nonexclusive, royalty-free license to publish or reproduce the published form of this contribution, or to allow others to do so, for U.S. Government purposes. Los Alamos National Laboratory requests that the publisher identify this article as work performed under the auspices of the U.S. Department of Energy. Los Alamos National Laboratory strongly supports academic freedom and a researcher's right to publish; as an institution, however, the Laboratory does not endorse the viewpoint of a publication or guarantee its technical correctness.

Los Alamos NATIONAL LABORATORY

Kessel: Continuous Integration and Developer Workflows

Richard Berger, Dr. techn. Dipl.-Ing.

March 20, 2026

Outline

Introduction

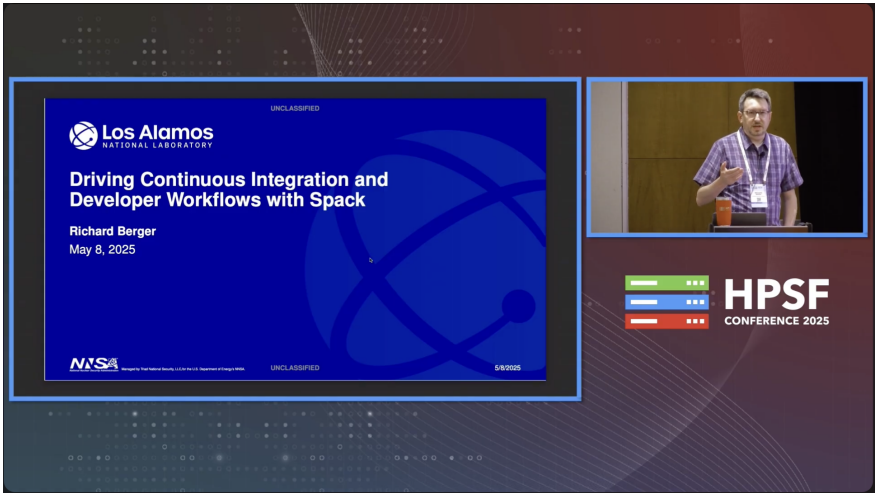
Why Kessel?

Kessel CLI and Workflows

Workflow Examples

Deployments

The Sequel to my HPSF Conference 2025 Talk



https://youtu.be/u73mESk0Q4E

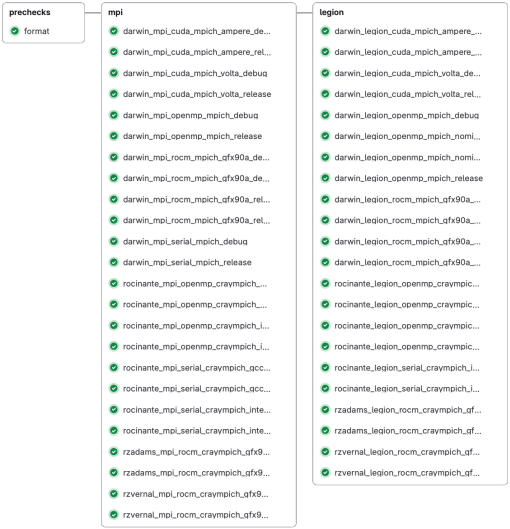
What are we building and testing?

- C/C++/Fortran HPC codes
- Research and Production codes
- Run on Tri-Lab (LANL, LLNL, Sandia) systems
  - LANL: Crossroads / Rocinante / Chicoma / Venado / Mission (soon) / Vision (soon)
  - LLNL: El Capitan / Tuolumne
- CPU-only and GPU-accelerated systems

What are we building and testing?

- Builds: Release, Debug
- Code Variants / Library Configurations
- Compilers: GCC, LLVM, Intel, NVIDIA, AMD, Cray
- Accelerations: OpenMP, CUDA, ROCm
- Communication:
  - MPI (MPICH, OpenMPI, CrayMPICH)
  - GASNET
- Hardware
  - GPUs: NVIDIA, AMD
  - Interconnects: HPE/Cray Slingshot 11, Infiniband

Continuous Integration (CI) Pipelines



- GitLab for Source Code Management
- Jacamar CI Runners for Tri-Lab systems
  - jobs can run via batch system

Developers

work on target systems,
use predefined configurations
and established workflows

DevOps

create configurations and pipelines
on target systems

Problem 1: Setting up software on a new HPC system can be hard

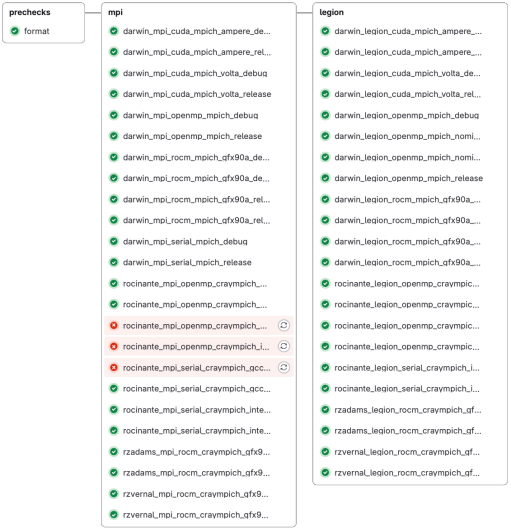
Your actual code
(Proprietary Libraries)
Open Source Libraries
Common Utilities
Vendor Drivers/Libraries
Operating System
Hardware

- We utilize powerful tools like Spack to help us manage this complexity
- It is still specialized knowledge to use these tools at scale
- **Dedicated experienced staff should be creating the right configuration on a given system, instead having X many people rediscover the same from *first principles*.**

Problem 2: Disconnect between Developer and CI workflows

Developers

"How do I recreate that CI job?"



Problem 2: Disconnect between Developers and CI workflows

Developer Workflows

- Manual setup scripts
- Ad-hoc commands
- Environment specific
- Hard to document

Continuous Integration Pipelines

- YAML configuration
- Different syntax
- Hard to test locally
- Diverges from dev workflow

Result

Developers and CI run different commands, leading to “works on my machine” problems and duplicate efforts.

Last year's presentation

Every GitLab CI job, on **all clusters**, prints a message like this at the beginning:

```
#####################################################################
To recreate this CI run, follow these steps:

ssh rzadams
cd /your/code/checkout
flux alloc -N=1 --time-limit=2h
source .gitlab/build_and_test.sh rzadams craympich-rocm-gfx942-gcc

See 'source .gitlab/build_and_test.sh -h' for more options.
#####################################################################
```

`build_and_test.sh` was a driver Shell script to orchestrate Spack and CMake

Outline

Introduction

Why Kessel?

Kessel CLI and Workflows

Workflow Examples

Deployments

Adoption of Spack-based Development Workflows

FleCSI	Ristra	XCAP
CMake	CMake	CMake
Spack **v1.x**	Spack **v1.x**	Spack **v1.x**
System Configs	System Configs	System Configs
Deployment Scripts	Deployment Scripts	Deployment Scripts
CI Setup and Scripts	CI Setup and Scripts	CI Setup and Scripts

Adoption of Spack-based Development Workflows

FleCSI	Ristra	XCAP
CMake	CMake	CMake
Spack **v1.x**	Spack **v1.x**	Spack **v1.x**
System Configs	System Configs	System Configs
Deployment Scripts	Deployment Scripts	Deployment Scripts
CI Setup and Scripts	CI Setup and Scripts	CI Setup and Scripts

unify and minimize maintenance

Adoption of Spack-based Development Workflows

FleCSI	Ristra	XCAP
CMake	CMake	CMake
Spack **v1.x**	Spack **v1.x**	Spack **v1.x**

Common Base Configurations

Common Deployment Logic

Common CLI and Workflows

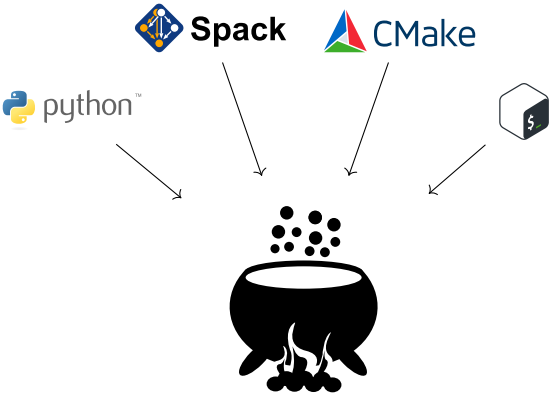
What is Kessel?

What is Kessel?

- ✗ **NOT** a package manager
- ✗ **NOT** a package manager *manager*
- ✗ **NOT** a build system
- ✗ **NOT** a build system generator
- ✗ **NOT** a new scripting language
- ✗ **NOT** yet another YAML/JSON/TOML layer

What is Kessel?

- A workflow orchestration tool for HPC software development
- Creates and drives CI/CD and developer workflows
- Establishes a unified interface across multiple projects and environments
- Driver and integration layer for build systems and package managers



Key Goals

⇒ Bridge the gap between CI pipelines and developer command-line workflows

⇒ Allow you to define these in a structured way by providing the necessary scaffolding

✔ **BYO (bring-your-own)** package manager

✔ **BYO (bring-your-own)** build system

✔ Use **Shell** and **Python** scripting to tie them together

⇒ Useful default workflows that can be extended / overwritten

Relationship between Kessel <-> Spack

- Kessel provides **Spack-based** workflows for continuous integration and software deployments
- Kessel utilizes and exposes Spack features and makes them more accessible
- Spack 1.x features have directly influenced how Kessel has evolved, reducing the need of complex features (e.g. generating templated YAML files, custom command wrappers) and letting it focus on a smaller feature set.
- Kessel evolved to be generic enough to support other package managers and build systems.
- E.g., Kessel's own CI uses Python/pip-based workflows.

kessel

Spack

Outline

Introduction

Why Kessel?

Kessel CLI and Workflows

Workflow Examples

Deployments

The Name: A double meaning

kessel

German: **cauldron** or **kettle**

Combine multiple ingredients to create your ~~magic potions~~ ***developer workflows***

The Name: A double meaning

kessel **run**

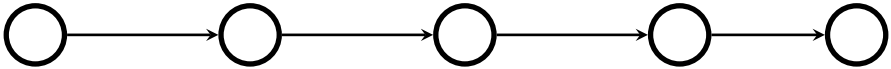
A playful reference to a popular SciFi movie

Short and simple way *of doing something that is usually long and complex.*

Workflows

kessel **run**

Executes a **workflow**.

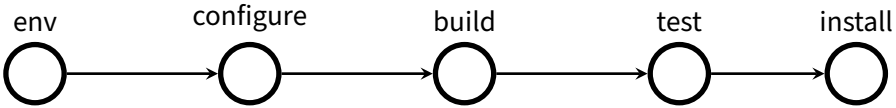


Workflows

kessel **run** → kessel **step** env
kessel **step** configure
kessel **step** build
kessel **step** test
kessel **step** install

Executes a **workflow**.

Workflows consist of **steps** run in order.

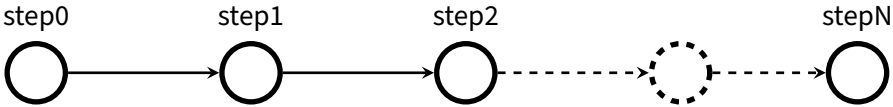


Steps

kessel run →

kessel step **<step0>**
kessel step **<step1>**
kessel step **<step2>**
kessel step **...**
kessel step **<stepN>**

Step names are dynamic and defined by workflow

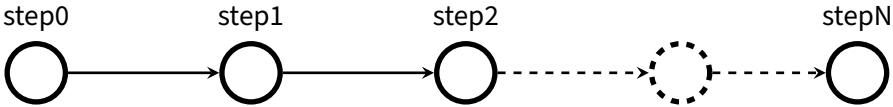


Command-line options

kessel run **<combined-options>** →

kessel step <step0> **<options0>**
kessel step <step1> **<options1>**
kessel step <step2> **<options2>**
kessel step ...
kessel step <stepN> **<optionsN>**

Step options are also run options.



Command-line Example

```
kessel run --env mpich-gcc flecsi+flog build_type=Debug
```

is the same as

```
kessel step env --env mpich-gcc flecsi+flog build_type=Debug
kessel step configure
kessel step build
kessel step test
```

Workflow File Layout

```
📁 myproject
└ 📁 .kessel
  └ 📁 workflows
    ├ 🗋 default.py
    ├ 🗋 format.py
    └ 📁 docs
      └ 🗋 __init__.py
```

- Each workflow is a Python module or package within the `.kessel/workflows` directory of a project
- To create these, you either use **`kessel init`** to create a project based on a template, or **`kessel create <name>`** to create one new workflow.
- You can edit the workflows with **`kessel edit [name]`**

Switching Workflows

```
$ kessel list
default
docs
format
```

```
$ kessel activate docs
$ kessel list
default
docs
format
```

- The **default** workflow is always active if it exists
- **`kessel list`** shows all workflows and highlights the active one.
- **`kessel activate <name>`** is used to switch to another workflow.

Workflow Definitions

```python
from kessel.workflows import Workflow

class Default(Workflow):
    steps = ["env", "build", "test"]

    def env(self, args):
        """Setup Environment"""
        # Environment logic here

    def build(self, args):
        """Build Project"""
        # Build logic here

    def test(self, args):
        """Run Tests"""
        # Test logic here
```

- Python classes that define a sequence of steps
- Steps are methods that are given the parsed command-line arguments:
  - method name ⇒ step name
  - method docstring ⇒ step label
- Derived from `Workflow` or other subclasses, allowing customization and mixing with other predefined workflows.

Steps

- Individual units of work within a workflow
- Execute shell commands or scripts (manipulate **parent** shell!)
- Maintain state across execution
- Can be run individually or as part of complete workflow

Step Execution

`kessel run`	Run all steps in sequence
`kessel step build`	Run a single step
`kessel status`	Check progress

Outline

Introduction

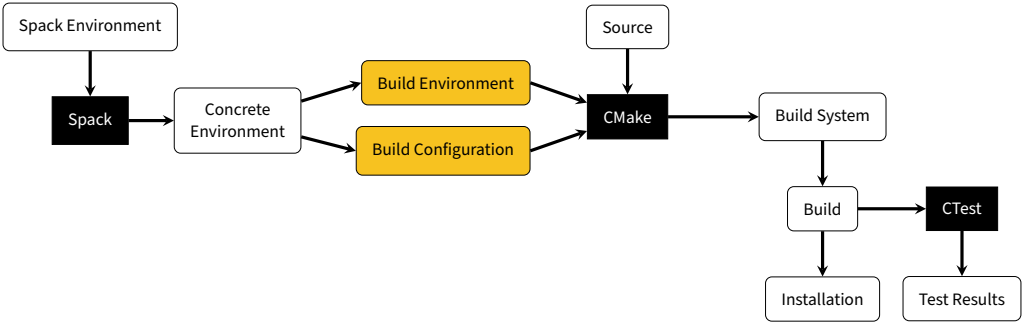
Why Kessel?

Kessel CLI and Workflows

Workflow Examples

Deployments

Built-in Workflows: Spack-based Environment and Build



Built-in Workflows: Spack-based Build Environment

```
from kessel.workflows import *
from kessel.workflows.base.cmake import CMake
from kessel.workflows.base.spack import BuildEnvironment

class Default(BuildEnvironment, CMake):
    steps = ["env", "configure", "build", "test", "install"]
    spack_env = environment("default")
    project_spec = environment("lammps")
```

Built-in Workflows: Spack-based Build Environment

1. Activates or creates a new Spack environment with a given name
2. Adds/Replaces a root spec in the environment
3. Registers a local Spack Package repository if it exists
4. Sets up `spack develop` for current project
5. Registers git mirrors for dependencies (e.g. git submodules)
6. Utilize and auto-push to per-user build cache
7. Concretizes and installs all build dependencies
8. Generates a CMake build folder based on the Spack package configuration

Built-in Workflows: Spack-based Build Environment

```
$ kessel run -h
usage: kessel run [-h] [-u {env,configure,build,test,install}]
                  [-e ENVIRONMENT] [-S SOURCE_DIR] [-B BUILD_DIR]
                  [-f] [-I INSTALL_DIR] ...

positional arguments:
  project_spec

options:
  -h, --help            show this help message and exit
  -u {env,configure,build,test,install}, --until {env,configure,build,test,ins
  -e ENVIRONMENT, --env ENVIRONMENT
  -S SOURCE_DIR, --source-dir SOURCE_DIR
  -B BUILD_DIR, --build-dir BUILD_DIR
  -f, --force           allow changes to Spack lockfile
  -I INSTALL_DIR, --install-dir INSTALL_DIR
```

Developer workflows

The simplest case:

```
kessel run --env myenv
```

Developer workflows

Let kessel do the environment setup, and then continue with regular CMake-based workflow.

```
kessel run -B mybuilddir --until configure
kessel build-env
cd mybuilddir
make -j $(nproc)
ctest
```

Developer workflows

Start with existing environment, customize it and continue with the rest of the workflow.

```
kessel run --until env
spack config edit # modify environment
spack concretize -f
kessel run
```

Outline

Introduction

Why Kessel?

Kessel CLI and Workflows

Workflow Examples

Deployments

Deployments

Pre-build software stacks for sharing and reuse.

Development and CI Environments

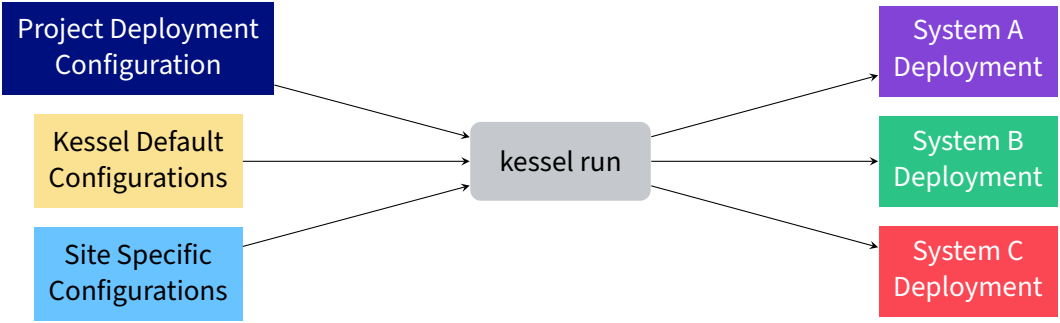
- development environments with pre-built dependencies
- includes build, run and test dependencies
- multiple configurations on heterogenous HPC clusters
- release and debug builds

Production Environments

- create final release environments of software packages
- only includes run dependencies

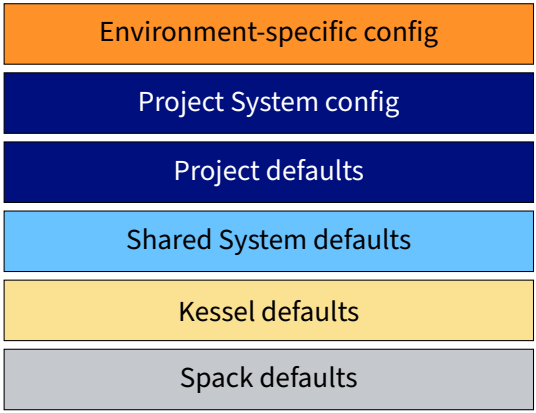
Spack-based Deployments

- Kessel provides a workflow to generate Spack-based deployments.
- This is one possible workflow that has worked for us at LANL.
- The general idea is that each system has a deployment and we can generate it from a shared configuration.



Spack-based Deployments: Configuration Scopes

This workflow uses Spack's layered configuration system:

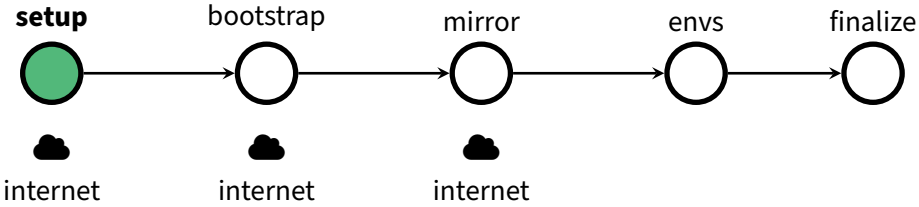


- Higher layers override lower layers
- Enables flexible, reusable configurations
- Kessel defaults provides templates that can be included for common configurations

Environment Example

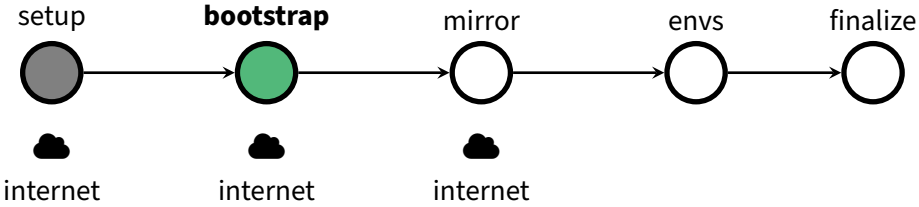
```yaml
spack:
  include:
  - $KESSEL_CONFIG_DIR/templates/gcc.yaml
  - $KESSEL_CONFIG_DIR/templates/mpich.yaml
  - $KESSEL_CONFIG_DIR/templates/cuda-ampere.yaml
  - name: kessel
    path: $KESSEL_CONFIG_DIR
  specs:
  - flecsi
  packages:
    flecsi:
      require:
      - build_type=Release +flog+hdf5+graphviz backend=legion %clang
    kokkos:
      require:
      - ~wrapper+serial~openmp+cuda cuda_arch=80 %cxx=clang
```

Spack Deployment Workflow



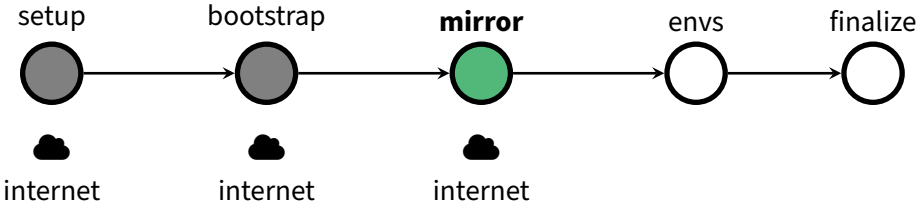
- generate deployment folder structure
- clone specific commit of spack, spack-packages
- render configuration and environments for system
- (optional) created copy of kessel
- create activate.sh to make spack (and if needed kessel) available

Spack Deployment Workflow



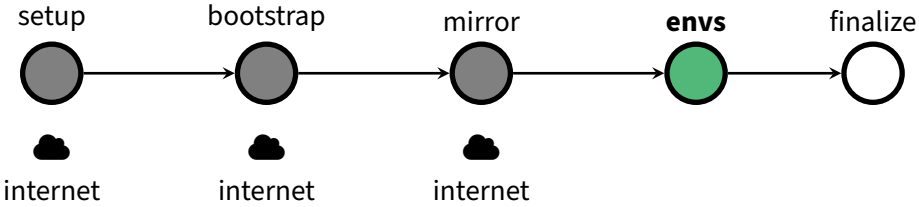
- install Spack dependencies

Spack Deployment Workflow



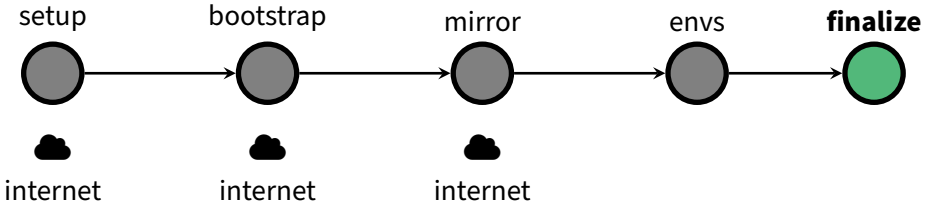
- concretize all environments
- download source code into deployment spack-mirror

Spack Deployment Workflow



- build out environments with or without roots

Spack Deployment Workflow



- cleanup and restore folder permissions

Utilizing a deployment

Typical GitLab CI usage pattern

.gitlab/kessel.sh

```
# finds the right deployment on each system and activates it
source /path/to/deployment/activate.sh
```

GitLab job script:

```
source .gitlab/kessel.sh
kessel run --env some-environment
```

Combined Result: Deployment + Spack-based Workflow

Every GitLab CI job, on **all clusters**, prints a message like this at the beginning:

```
#####################################################################
To recreate this CI run, follow these steps:

ssh rzadams
cd /your/code/checkout
flux alloc -N=1 --time-limit=2h
source .gitlab/kessel.sh
kessel run --env craympich-rocm-gfx942-gcc

See 'kessel run -h' for more options.
#####################################################################
```

Conclusion

- Kessel turns multiple steps into a single, reproducible command - making the CI-dev gap disappear.
- Generic tool, but provides first class Spack support
- New abstraction not just useful for humans, but also AI tooling
- Adoption of kessel is ongoing in multiple LANL ASC code teams
- Shared DevOps efforts to maintain common HPC cluster base configurations

Acknowledgments

- Advanced Simulation and Computation (ASC) program:
  - FleCSI
  - Ristra
  - Cross-cutting Capabilities Project (XCAP)
  - Integrated Codes (IC) DevOps
- Davis Herring, FleCSI lead architect: Challenging and thoughtful criticism, motivated me to create the project and proposed the name.
- Andres Yague Lopez, for being a willing Co-Conspirator, early tester and contributor

Questions

GitHub: https://github.com/lanl/kessel
Contact: rberger@lanl.gov